

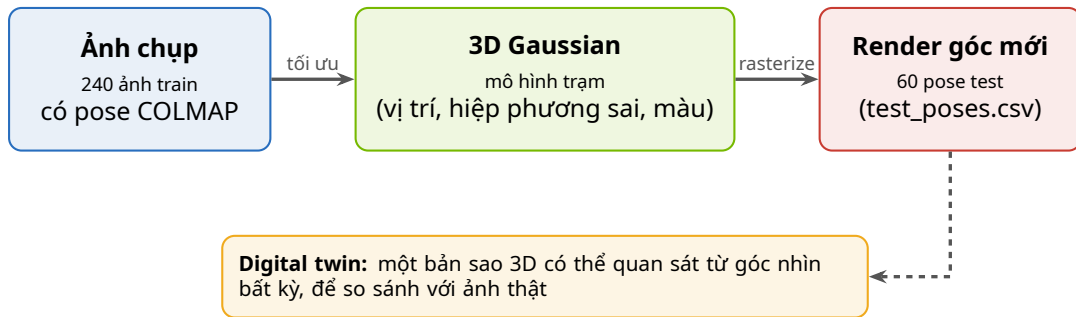
Tái tạo trạm BTS bằng Render 3D

Từ CUDA rasterizer đến SADGS — digital twin cho trạm viễn thông

Thuyết trình 30 phút

Ngày 3 tháng 10 năm 2026

Bài toán: tái tạo trạm BTS thành digital twin



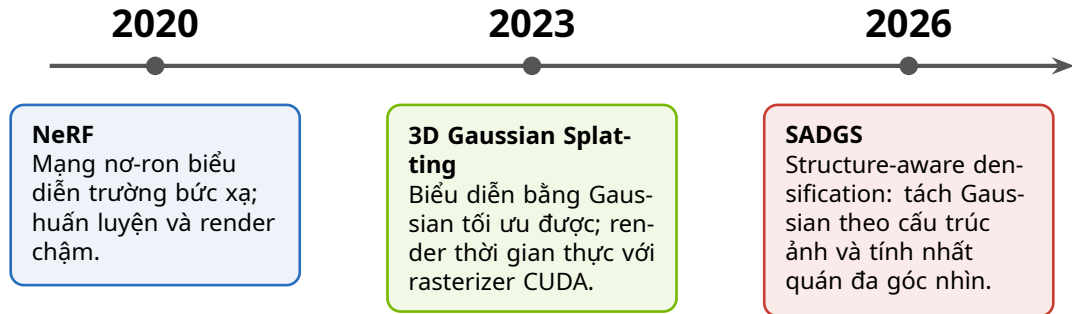
- Từ vài trăm ảnh chụp trạm, dựng mô hình 3D để **render góc nhìn mới**.
- Dữ liệu cuộc thi VAI_NVS_DATA_ROUND2, scene HCM0539: 240 ảnh train, 60 ảnh test.
- Lỗi kỹ thuật: 3D Gaussian Splatting, cải tiến bằng SADGS.

Dữ liệu một scene: 240 ảnh train, 60 ảnh test



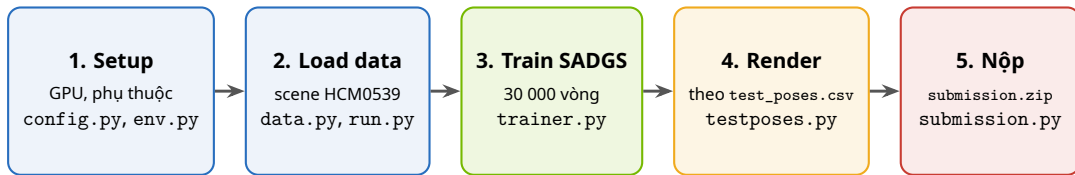
- **train/**: 240 ảnh kèm camera COLMAP (**sparse/0/**) để tối ưu mô hình.
- **test/**: 60 ảnh có ground truth để chấm; **test_poses.csv** là 60 pose cần render để nộp.
- Scene được đặt tên theo thư mục cha (HCM0539), nên **submission.zip** có cấu trúc HCM0539/0001.png.

Bối cảnh: NeRF (2020), 3DGS (2023), SADGS (2026)



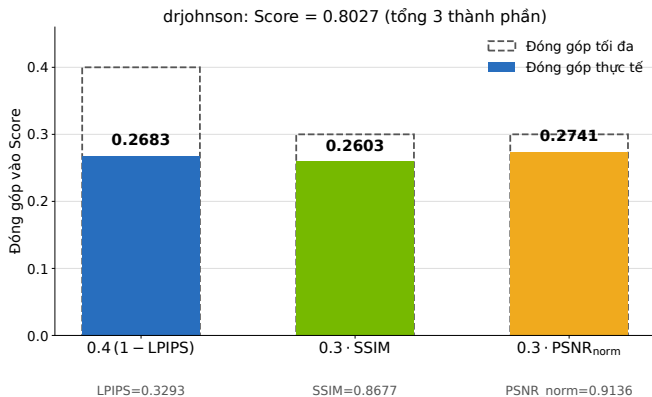
- Mốc 2020 (NeRF) và 2023 (3DGS) là kiến thức chung của lĩnh vực, không lấy từ tài liệu dự án.
- SADGS (Lyu et al., SIGGRAPH 2026): giữ khung 3DGS, thay tiêu chí tách Gaussian bằng cấu trúc ảnh đa tỉ lệ.

Pipeline thực tế: 5 bước từ setup đến submission.zip



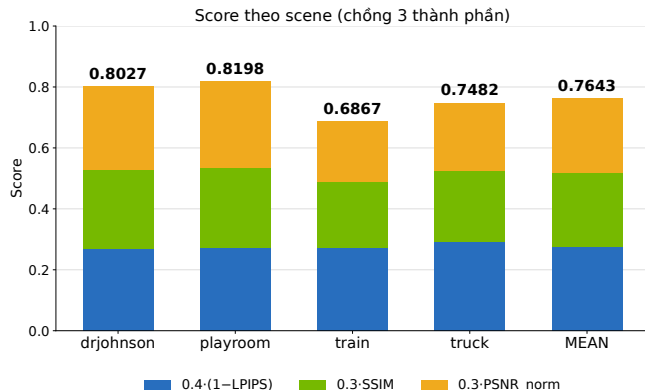
- Cấu hình `iterations=30000`, render ở độ phân giải gốc (`submission_resolution=1`).
- Sau mỗi scene: lưu `.ply` lên Drive trước, rồi mới render test pose để tránh mất model khi phiên bị ngắt.

Công thức điểm: $\text{Score} = 0.4(1 - \text{LPIPS}) + 0.3 \text{ SSIM} + 0.3 \text{ PSNR}_{\text{norm}}$



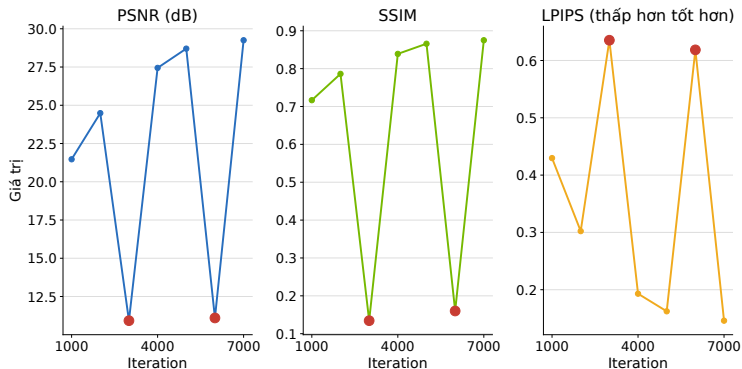
- $\text{PSNR}_{\text{norm}} = \text{clamp}(\text{PSNR} / 30, 0, 1)$; LPIPS càng thấp càng tốt nên dùng $1 - \text{LPIPS}$.
- Nguồn: `pipeline/score.py`. Nét đứt là đóng góp tối đa của từng thành phần.

Kết quả theo scene: Score trung bình 0.7643



- playroom cao nhất (0.8198); train thấp nhất (0.6867, PSNR 19.75 dB).
- Lưu ý: leaderboard.csv ghi `iters=7000` cho cả 4 scene, khác cấu hình 30 000 vòng; cần xác nhận trước khi trình bày.

Diễn biến huấn luyện (drjohnson): PSNR, SSIM, LPIPS theo vòng lặp



Đỏ: mốc 3000 và 6000 có chỉ số rất bất thường (cần kiểm tra)

- Đánh giá live mỗi 1000 vòng, không khớp leaderboard chính thức (7000 vòng: live PSNR 29.25 dB, bảng cuối 27.41 dB).
- Mốc 3000 (PSNR 10.92) và 6000 (PSNR 11.10) rất bất thường, SSIM cũng sụp xuống dưới 0.2; **cần kiểm tra log trước khi kết luận.**

Vì sao digital twin quan trọng và tiêu chí đánh giá

Vì sao digital twin quan trọng

- Kiểm tra trạm từ xa, không cần lên cột.
- Lưu hiện trạng để so sánh theo thời gian.

Tiêu chí đánh giá

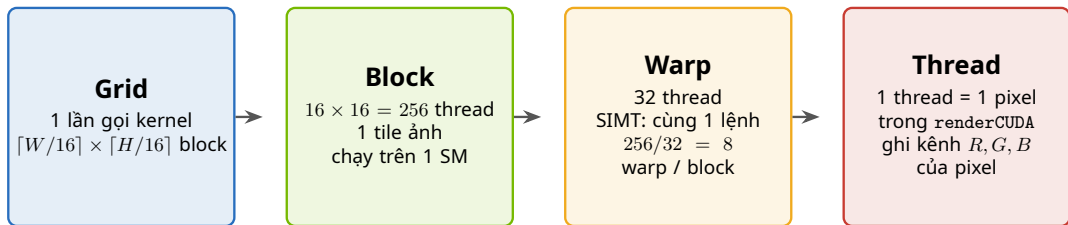
- Chất lượng hình ảnh: PSNR, SSIM, LPIPS.
- Khả năng render góc lạ (60 pose chưa thấy).

$$\text{Score} = 0.4 \cdot (1 - \text{LPIPS}) + 0.3 \cdot \text{SSIM} + 0.3 \cdot \text{PSNR}_{\text{norm}}$$

gộp chất lượng cảm nhận, cấu trúc và độ chính xác điểm ảnh

- Chất lượng hình ảnh đo bằng PSNR, SSIM, LPIPS trên ảnh hold-out.
- Khả năng render góc lạ đo trên 60 pose mà mô hình chưa từng thấy.

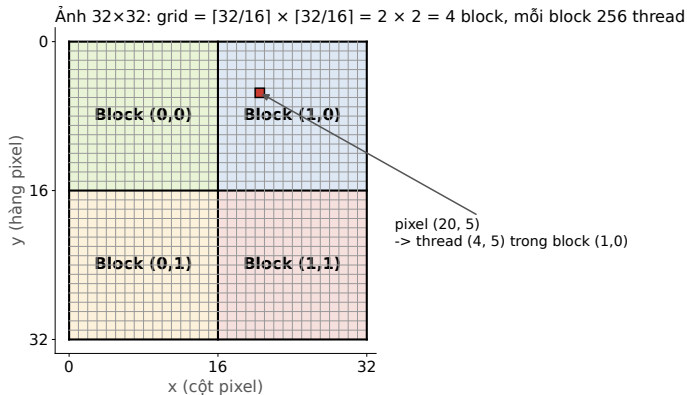
GPU và mô hình grid-block-warp-thread



Phân cấp: lưới gồm nhiều block, block gồm nhiều warp, warp gồm 32 thread

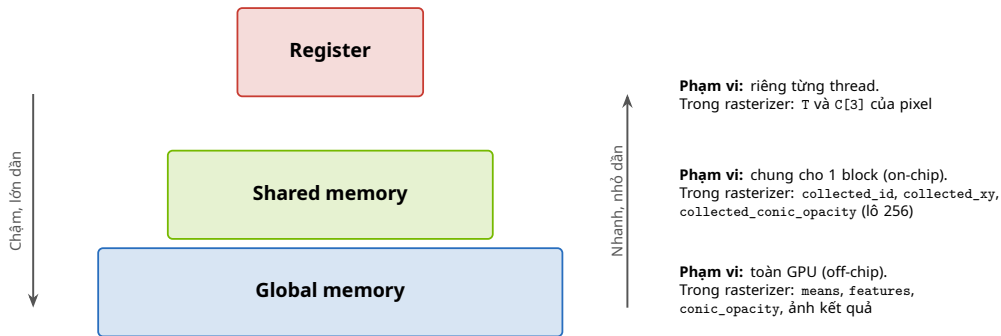
- Một lần gọi kernel tạo **lưới** gồm $\lceil W/16 \rceil \times \lceil H/16 \rceil$ block.
- Mỗi block có $16 \times 16 = 256$ thread, chạy trên một SM; các thread chia thành $256/32 = 8$ warp.
- Thread trong renderCUDA xử lý đúng một pixel.

Ảnh xạ pixel $\rightarrow$ thread: một block là một tile 16×16



- Pixel (p_x, p_y) thuộc block $(\lfloor p_x/16 \rfloor, \lfloor p_y/16 \rfloor)$, thread $(p_x \bmod 16, p_y \bmod 16)$.
- Lưới tile: `dim3 tile_grid((W+15)/16, (H+15)/16, 1)`. Ảnh 32×32 cho lưới 2×2 .

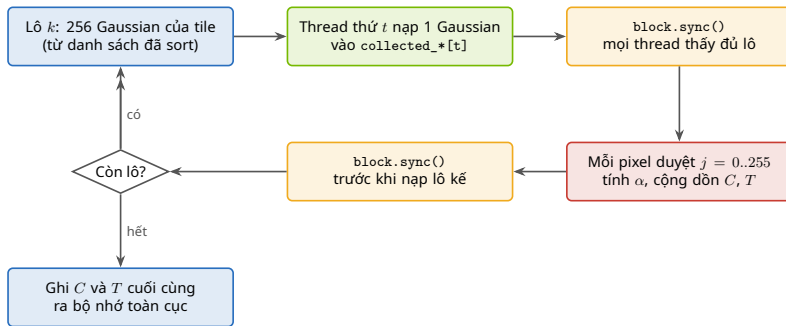
Bộ nhớ CUDA: register $\rightarrow$ shared $\rightarrow$ global



Chỉ so sánh định tính. Không quy định số băng thông hay độ trễ cụ thể.

- Càng gần thread thì càng nhanh nhưng dung lượng càng nhỏ; cần đặt dữ liệu dùng lặp lại vào tầng gần hơn.
- Shared memory do cả block dùng chung, nên một lần nạp phục vụ được 256 pixel.

Vì sao nạp lô 256 Gaussian vào shared memory?



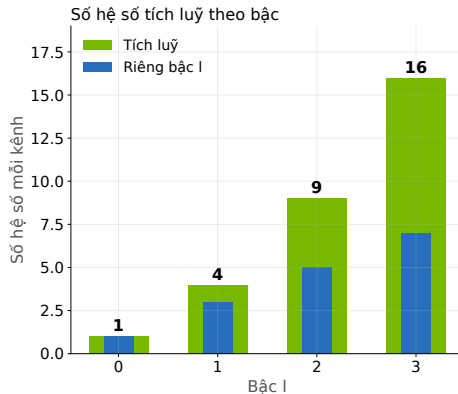
Thread đã xong (done) vẫn phải tham gia nạp và `block.sync()`: nếu không, block bị treo.

- Mỗi lô tối đa 256 phần tử, bằng số thread của block (`BLOCK_SIZE=256`): thread t nạp Gaussian thứ t .
- `collected_xy`, `collected_conic_opacity[256]` là mảng chung; `block.sync()` đảm bảo mọi thread cùng thấy lô.

Spherical Harmonics: số hệ số theo bậc

Bảng hệ số SH thực (mỗi kênh R, G, B)

Bậc l	Số hệ số $(2l+1)$	m	Tích lũy (đến bậc l)
0	1	0	1
1	3	-1, 0, 1	4
2	5	-2, -1, 0, 1, 2	9
3	7	-3, -2, -1, 0, 1, 2, 3	16



- Bậc l có $2l + 1$ hệ số ($m = -l, \dots, l$): 1, 3, 5, 7 cho $l = 0..3$.
- Tích lũy đến bậc 3: $1 + 3 + 5 + 7 = 16$ hệ số cho mỗi kênh R, G, B , tức 48 số thực mỗi Gaussian.

computeColorFromSH: từ hướng nhìn đến màu

1. Hướng nhìn

$\mathbf{d} = \frac{\boldsymbol{\mu} - \mathbf{o}_{cam}}{\|\boldsymbol{\mu} - \mathbf{o}_{cam}\|}$
 $(x, y, z) = (d_x, d_y, d_z)$
 $\mathbf{dir} = \mathbf{pos} - \mathbf{campos};$
 $\mathbf{dir} = \mathbf{dir} / \text{length}(\mathbf{dir});$

2. Hàm cơ sở

Bậc 1: $-y, z, -x$
Bậc 2: $xy, yz, 2z^2 - x^2 - y^2, xz, x^2 - y^2$
Bậc 3: 7 đa thức bậc 3 của (x, y, z)

3. Cộng dồn

$\mathbf{c} = C_0 \text{sh}_{dc}$
 $\mathbf{c} += \sum_{\ell, m} C_{\ell}^m Y_{\ell}^m(\mathbf{d}) \text{sh}_k$
cho từng bậc $\text{deg} > 0, \text{deg} > 1, \text{deg} > 2$

4. Offset và clamp

$\mathbf{c} \leftarrow \mathbf{c} + 0.5$
 $\mathbf{c}_{final} = \max(\mathbf{c}, 0)$
lưu $\text{clamped}[3i+k] = (c_k < 0)$ cho backward

Nguồn: forward.cu dòng 24-76 (computeColorFromSH). Mỗi Gaussian có 16 hệ số SH mỗi kênh.

- Hướng nhìn $\mathbf{d}$ được chuẩn hoá trước khi tính hàm cơ sở.
- Bậc 0 dùng dc riêng; các bậc cao hơn chỉ được cộng vào khi deg đủ lớn.

Hằng số SH thực và đa thức bậc 1

Hằng số chuẩn hoá (auxiliary.h)

$$C_0 = \sqrt{\frac{1}{4\pi}} = 0.28209479 \dots \quad C_1 = \sqrt{\frac{3}{4\pi}} = 0.48860251 \dots$$

$$C_2 = \{ 1.09254843, -1.09254843, 0.31539157, -1.09254843, 0.54627422 \}$$

$$C_3 = \{ -0.59004359, 2.89061144, -0.45704580, 0.37317633, -0.45704580, 1.44530572, \dots \}$$

Đa thức bậc 1 (ví dụ công thức)

$$\mathbf{c} += -C_1 y \text{sh}_0 + C_1 z \text{sh}_1 - C_1 x \text{sh}_2$$

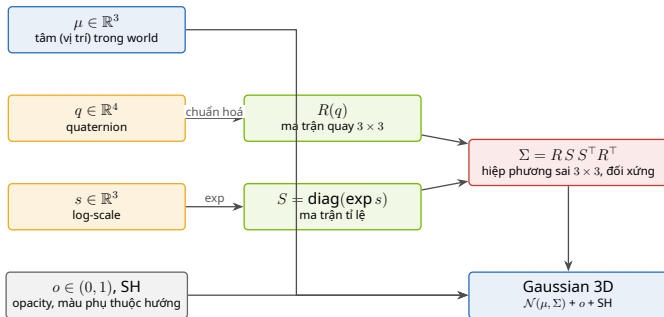
$$\text{Lấy } \mathbf{d} = (0.6, 0, 0.8), \|\mathbf{d}\| = 1:$$

$$\mathbf{c} += 0.3909 \text{sh}_1 - 0.2932 \text{sh}_2$$

Dấu của C_2 và C_3 lấy đúng như nguồn; số thập phân làm tròn 8 chữ số.

- C_0, C_1 là các hệ số chuẩn hoá của hàm cầu điều hoà thực bậc 0 và 1 (auxiliary.h).
- Ví dụ: với $\mathbf{d} = (0.6, 0, 0.8)$ chỉ còn hai số hạng bậc 1 khác 0.

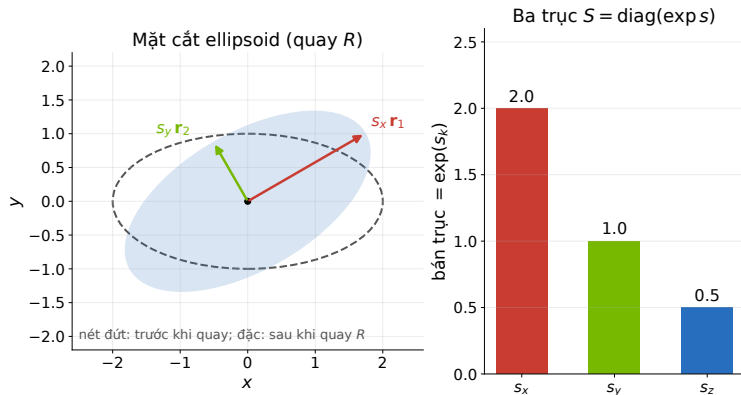
Gaussian 3D: tham số và hiệp phương sai



Ghi chú: Σ là hiệp phương sai của Gaussian trong world-space.
Code CUDA nhận q đã chuẩn hoá và s đã qua exp từ phía Python.

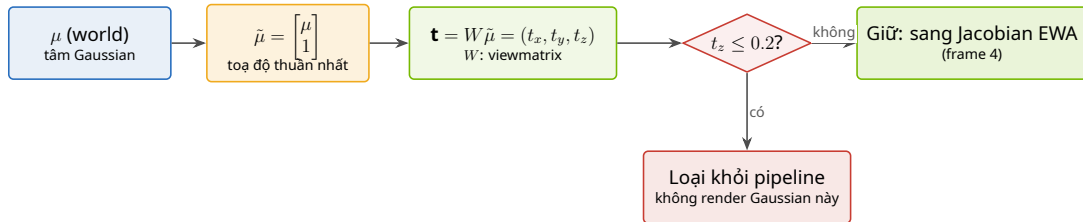
- Mỗi Gaussian có $\mu \in \mathbb{R}^3$, quaternion q , log-scale s , opacity o và hệ số SH.
- $\Sigma = R S S^T R^T$ với $S = \text{diag}(\exp s)$; Σ đối xứng, chỉ lưu 6 phần tử trên (cov3D).
- Code: `computeCov3D` (`forward.cu:135–169`) dựng $M = SR$, $\Sigma = M^T M$.

Ellipsoid 3D: trục, hướng và tỉ lệ



- Trục chính của ellipsoid là các cột $\mathbf{r}_k$ của R ; bán trục là $s_k = \exp(\cdot)$ tương ứng.
- Ví dụ: $S = \text{diag}(2.0, 1.0, 0.5)$, quay 30° quanh trục z ; mặt cắt $z = 0$ là ellipse nét đặc.
- Mức 1σ : $\{\mathbf{x} : \mathbf{x}^\top \Sigma^{-1} \mathbf{x} = 1\}$ có bán trục đúng bằng s_k .

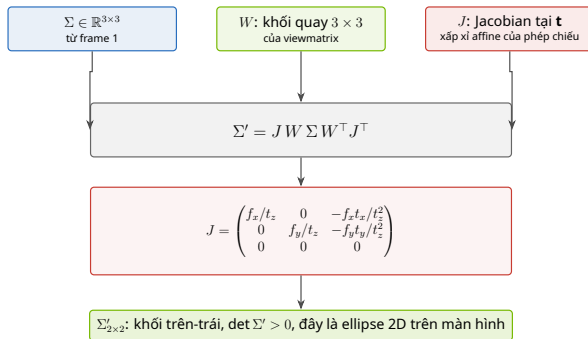
Biến đổi sang hệ camera và cắt cận



Code: `in_frustum(auxiliary.h:167)`, $p_{view,z} \leq 0.2$ thì bỏ Gaussian.
Ngưỡng 0.2 là khoảng cách mặt cắt gần (near plane) trong camera-space.

- $\tilde{\mu} = \begin{bmatrix} \mu \\ 1 \end{bmatrix}$, $\mathbf{t} = W\tilde{\mu} = (t_x, t_y, t_z)$, với W là viewmatrix (world $\rightarrow$ camera).
- Gaussian bị loại nếu $t_z \leq 0.2$ (`in_frustum`, `auxiliary.h:167`).
- Kiểm tra theo NDC đã bị vô hiệu hoá trong code; chỉ còn cắt theo mặt cắt gần.

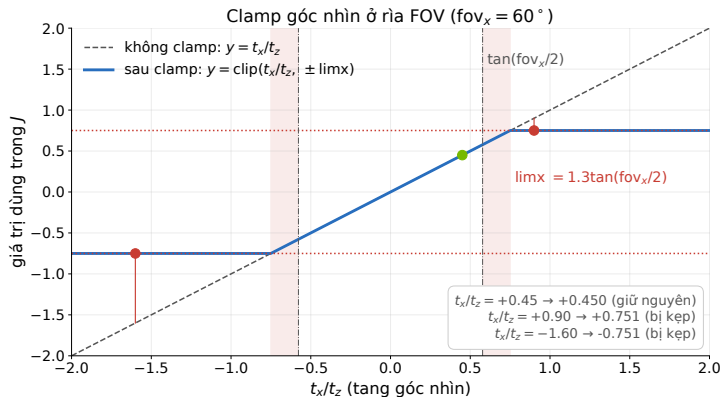
Jacobian EWA và hiệp phương sai 2D



Code: `computeCov2D (forward .cu:79-130)`, dùng quy ước transpose của GLM (cột-chính).

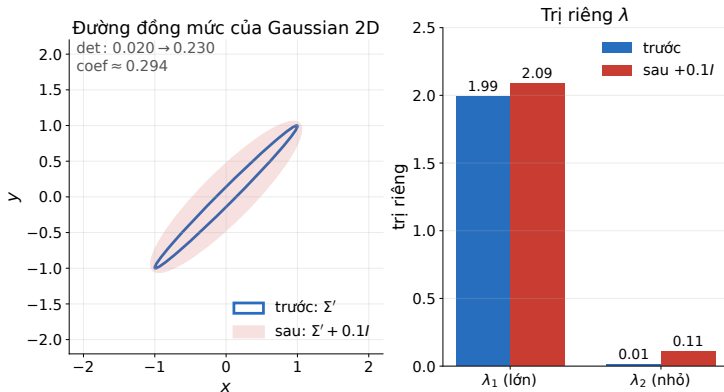
- J là đạo hàm riêng của phép chiếu $(t_x, t_y, t_z) \mapsto (f_x t_x / t_z, f_y t_y / t_z)$ tại $\mathbf{t}$ (EWA, Zwicker 2002).
- Hàng cuối của J bằng 0 nên chỉ giữ khối 2×2 trên-trái của Σ' .
- J dùng t_x, t_y đã kẹp ở rìa FOV (frame sau).

Clamp góc nhìn ở rìa FOV



- $\text{limx} = 1.3 \tan(\text{fov}_x/2)$, $t_x \leftarrow \text{clip}(t_x/t_z, \pm \text{limx}) t_z$ (forward.cu:86–93); tương tự cho y .
- Ví dụ $\text{fov}_x = 60^\circ$: $\tan 30^\circ \approx 0.577$, $\text{limx} \approx 0.751$ (vùng đệm khoảng 30% ngoài biên).
- Mục đích: giữ J hữu hạn với điểm ở rìa FOV, không để $1/t_z^2$ làm biến dạng ellipse.

Anti-aliasing: cộng $0.1 /$ vào Σ'



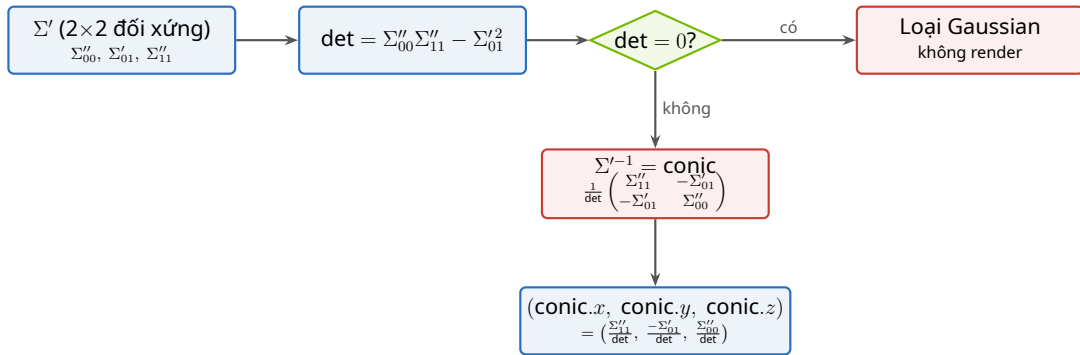
- `kernel_size= 0.1` (`forward.cu:118`): $\Sigma'' = \Sigma' + 0.1I$. Ví dụ: $\Sigma' = \begin{bmatrix} 1 & 0.99 \\ 0.99 & 1 \end{bmatrix}$, $\det: 0.020 \rightarrow 0.230$.
- Trị riêng: λ đi từ 1.99/0.01 thành 2.09/0.11; trục nhỏ tăng 11 lần, Gaussian không còn mỏng dưới 1 pixel.
- Bù năng lượng: $\text{coef} = \sqrt{\det_0 / \det_1} \approx 0.294$ nhân vào opacity.

Lưu ý: 0.3 (3DGS gốc) và 0.1 (SADGS)

Tiêu chí	3DGS gốc Kerbl 2023	SADGS forward.cu	SADGS backward.cu
Hằng số low-pass cộng vào Σ' (đường chéo)	0.3	0.1	0.3
Hệ số bù coef nhân opacity	không	có	chưa kiểm
Trần alpha	0.99	0.99	chưa kiểm
Hệ số giới hạn FOV 1.3	—	1.3	1.3

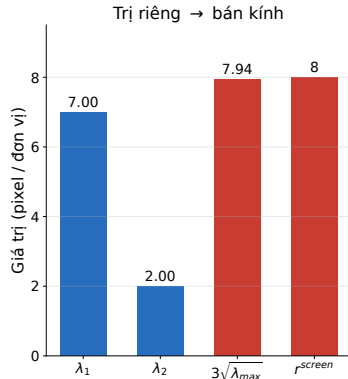
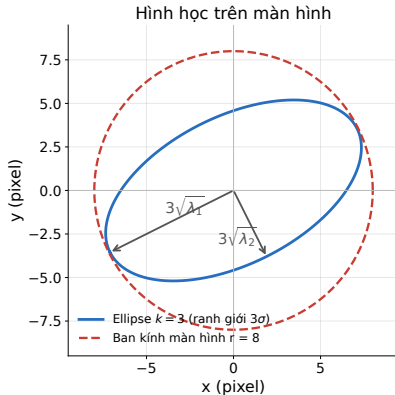
- Forward dùng 0.1 (forward.cu:118) nhưng backward dùng 0.3 (backward.cu:199–201).
- Hệ quả: gradient không phải đạo hàm đúng của hàm forward đang chạy; đây là lịch đã biết, chưa sửa trong phần này.
- Cần thống nhất hằng số giữa forward và backward, rồi kiểm tra bằng gradient check số.

preprocessCUDA: nghịch đảo hiệp phương sai



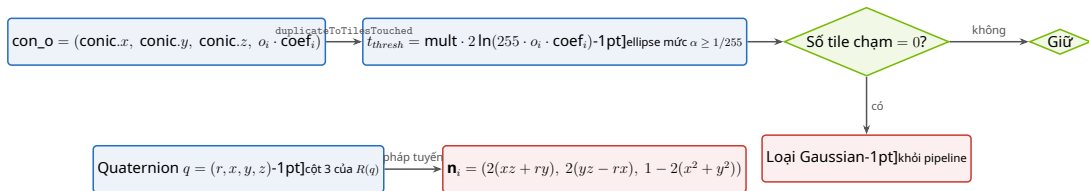
- $\det = \Sigma''_{00}\Sigma''_{11} - \Sigma'^2_{01}$; nếu $\det = 0$ thì loại Gaussian khỏi pipeline.
- $\Sigma'^{-1} = \text{conic} = \frac{1}{\det} \begin{pmatrix} \Sigma''_{11} & -\Sigma'_{01} \\ -\Sigma'_{01} & \Sigma''_{00} \end{pmatrix}$, lưu thành (conic.x, conic.y, conic.z).
- Ma trận nghịch đảo được tính một lần trong preprocessCUDA để renderCUDA chỉ cần nhân dạng bậc hai.

Bán kính màn hình qua trị riêng



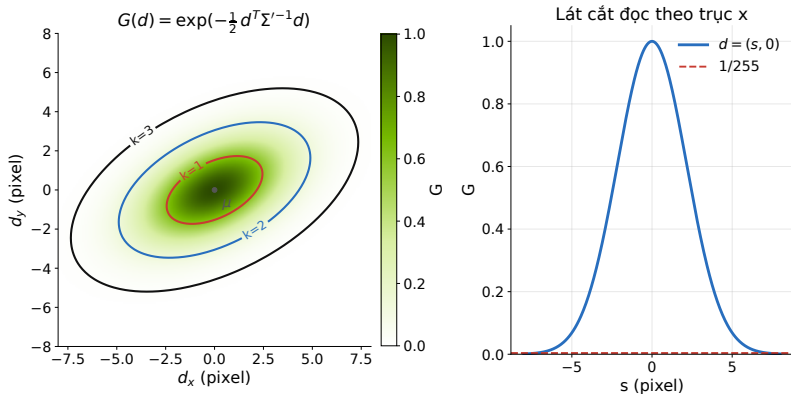
- $\text{mid} = \frac{\Sigma''_{00} + \Sigma''_{11}}{2}$, $\lambda_{1,2} = \text{mid} \pm \sqrt{\max(0.1, \text{mid}^2 - \det)}$.
- $r^{\text{screen}} = \lceil 3\sqrt{\max(\lambda_1, \lambda_2)} \rceil$: ellipse mức 3σ nằm gọn trong hình tròn bán kính r .
- Ví dụ minh họa $\Sigma' = \begin{pmatrix} 6 & 2 \\ 2 & 3 \end{pmatrix}$: $\lambda_1 = 7$, $\lambda_2 = 2$, $r = \lceil 3\sqrt{7} \rceil = 8$.

Opacity hiệu dụng, lọc theo tile và pháp tuyến



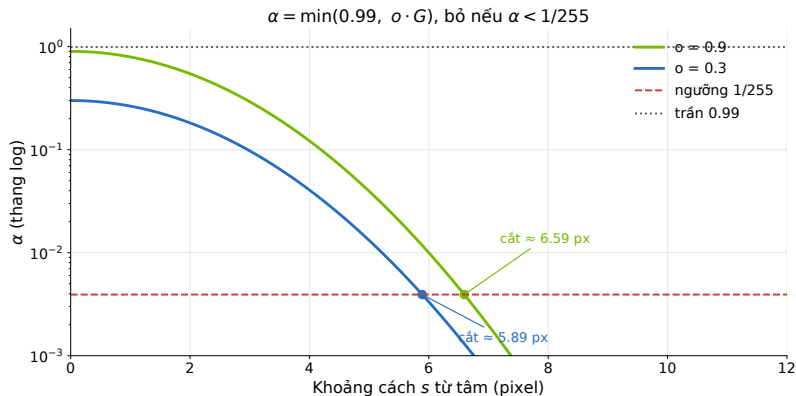
- $con_o.w = o_i \cdot coef_i$ là opacity hiệu dụng; ngưỡng $t_{thresh} = mult \cdot 2 \ln(255 \cdot o_i \cdot coef_i)$ xác định các tile mà ellipse $\alpha \geq 1/255$ chạm tới.
- Gaussian không chạm tile nào bị loại khỏi pipeline.
- Pháp tuyến $\mathbf{n}_i$ lấy từ cột thứ ba của $R(q)$ (xấp xỉ cho Gaussian phẳng, kiểu StructGS/2DGS).

Độ mờ Gaussian 2D tại pixel



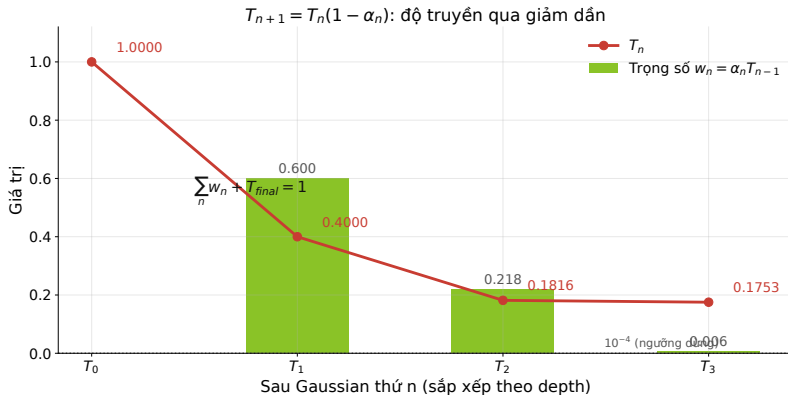
- Với $\mathbf{d} = \mathbf{x}y_i - \mathbf{x}_{pix}$: power = $-\frac{1}{2} \mathbf{d}^T \Sigma'^{-1} \mathbf{d}$ và $G(\mathbf{d}) = \exp(\text{power})$.
- Nếu power > 0 (sai số làm tròn làm conic mất tính xác định dương) thì bỏ qua pixel.
- Rasterizer không chuẩn hoá theo $1/(2\pi\sqrt{\det \Sigma'})$: $G(\mathbf{0}) = 1$ và opacity đóng vai trò biên độ.

Alpha của Gaussian tại pixel



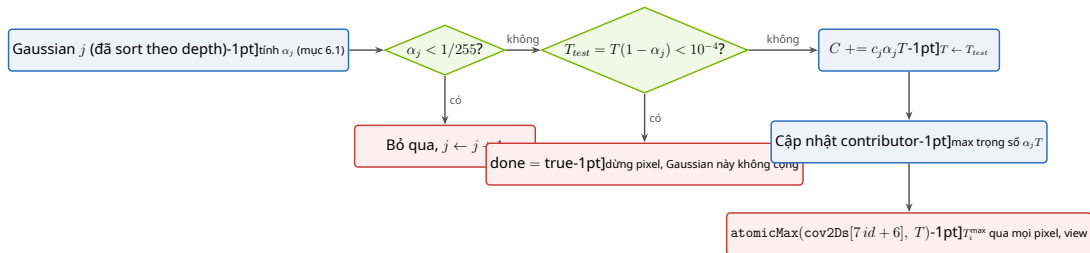
- $\alpha_i = \min(0.99, o_i \text{coef}_i G(\mathbf{d}))$; trần 0.99 giữ số học ổn định, như code 3DGS gốc.
- Nếu $\alpha_i < 1/255$ thì bỏ qua: đóng góp dưới độ phân giải 8-bit của một mức xám.
- Đường cong minh họa dùng $\Sigma' = \text{diag}(4, 4)$, $o \in \{0.9, 0.3\}$; điểm cắt $1/255$ nằm ở khoảng cách khác nhau.

Alpha compositing front-to-back



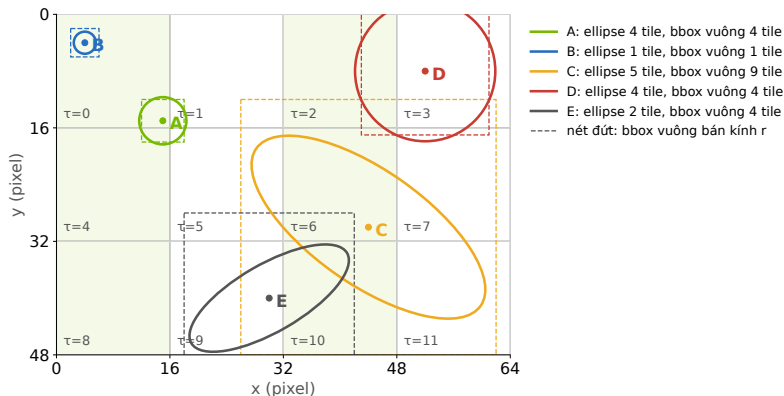
- $T_0 = 1$, $T_i = T_{i-1}(1 - \alpha_i)$; màu $C = \sum_i c_i \alpha_i T_{i-1}$, cộng màu *trước* khi cập nhật T .
- Cuối cùng $C_{final} = C + T_{final}$ **bg**; tổng trọng số $\sum_i \alpha_i T_{i-1} + T_{final} = 1$.
- Ví dụ 3 Gaussian (mục 10.2 của nguồn): $C_{final} \approx (0.6175, 0.2359, 0.0239)$.

Dừng sớm, contributor và $T^{\max}$



- Nếu $T_{test} = T(1 - \alpha_j) < 10^{-4}$ thì dừng pixel; ghi chú: nguồn ghi $T_{test} < 10^{-4}$, kiểm tra trước khi cộng màu.
- `atomicMax` trên `cov2Ds` lưu $T_i^{\max}$ qua mọi pixel và view; số học float dương cho phép so sánh bit như int.
- `max_weight` và `max_id` theo dõi Gaussian đóng góp trọng số α_jT lớn nhất cho mỗi pixel.

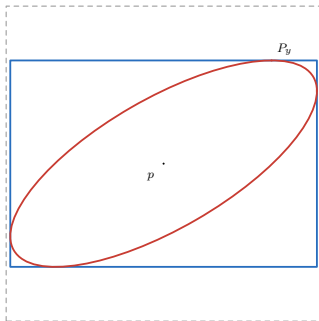
Lưới tile và số tile phủ



- Tile kích thước 16×16 pixel: $\text{grid}_x = \lceil W/16 \rceil$, $\text{grid}_y = \lceil H/16 \rceil$; ảnh 64×48 cho lưới $4 \times 3 = 12$ tile.
- tiles_touched_i là số tile mà ellipse mức t của Gaussian i thực sự cắt.
- Ví dụ: C chạm 5 tile, trong khi bbox vuông cùng bán kính chứa tới 9 tile.

SnugBox: ellipse mức đồng mức

bbox vuông, bán kính $r = 12$



bbox ôm khít: $W_x = 11.69, W_y = 7.86$

Ellipse mức t :

$$(\mathbf{x} - \mathbf{p})^\top \Sigma^{-1} (\mathbf{x} - \mathbf{p}) = t$$

$$\begin{aligned} t &= \text{mult} \cdot 2 \ln(255 \alpha) \\ &= 2 \ln(255 \cdot 0.9) \approx 10.87 \end{aligned}$$

Tile được tính khi ellipse *cắt* tile,
không phải khi tile nằm trong bbox vuông.

- Ngưỡng đóng góp $1/255$: $t = \text{mult} \cdot 2 \ln(255 \alpha)$ (rasterizer_impl.cu, duplicateToTilesTouched).
- Ellipse $(\mathbf{x} - \mathbf{p})^\top \Sigma^{-1} (\mathbf{x} - \mathbf{p}) = t$ được ôm bằng bbox trực song song (tight box) qua x_{term} , y_{term} .
- Bbox vuông bán kính r là bản cũ đã bị comment, chỉ còn là tham chiếu lịch sử.

Prefix sum và offset

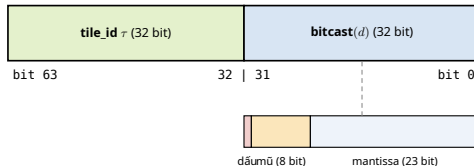
	A (i=0)	B (i=1)
tiles_touched	4	1
offset (inclusive)	4	5
off (exclusive)	0	4

$$L = \text{num_rendered} = \text{offset}[P-1] = 5$$

- `cub::DeviceScan::InclusiveSum`: $\text{offset}_i = \sum_{j \leq i} \text{tiles_touched}_j$.
- Chỉ số ghi của Gaussian i là $\text{off}_i = \text{offset}_{i-1}$ (với $\text{off}_0 = 0$), tức prefix sum loại trừ.
- Số cặp (Gaussian, tile) cần cấp phát: $L = \text{offset}_{P-1} = 5$.

Khoá sắp xếp 64 bit

$$\text{key} = (\tau \ll 32) \mid \text{bitcast}_{32}(d)$$



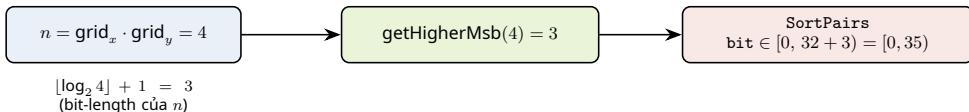
$$\tau = 2, d = 2.0 \Rightarrow 0x0000000024000000$$

$0x00000000 \mid 0x40000000$, với $d = 2.0$: dấu = 0, mũ = 10000000_2 , mantissa = 0

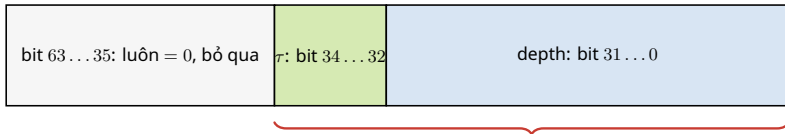
So sánh số nguyên 64 bit: τ quyết định trước, d quyết định khi τ bằng nhau.
Thứ tự theo d đúng chỉ khi $d \geq 0$ (near-plane: $d > 0.2$).

- $\text{key} = (\tau \ll 32) \mid \text{bitcast}_{32}(d)$, với τ là chỉ số tile và d là depth camera-space.
- Ví dụ: $\tau = 2, d = 2.0 \Rightarrow 0x0000000024000000$; $d = 1.0 \Rightarrow 0x3F800000$.
- Thứ tự theo d đúng chỉ khi $d \geq 0$; near-plane $z > 0.2$ đảm bảo điều này.

Radix sort toàn cục và getHigherMsb



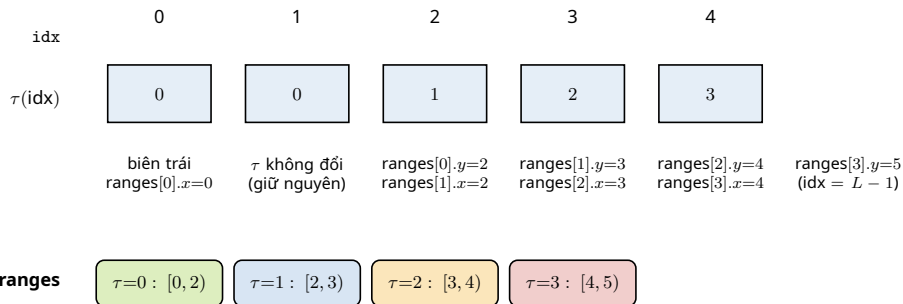
Khóa 64 bit (bit 63 bên trái, bit 0 bên phải):



sort trên $[0, 35)$: 32 bit depth + 3 bit tile

- Chỉ sort các bit $[0, 32 + \text{bit})$, với $\text{bit} = \text{getHigherMsb}(\text{grid}_x \cdot \text{grid}_y)$.
- $\text{getHigherMsb}(n) = \lfloor \log_2 n \rfloor + 1$; với $n = 4$ thì $\text{bit} = 3$, sort trên $[0, 35)$.
- Các bit $[35, 64)$ luôn bằng 0 nên bị bỏ qua mà không ảnh hưởng kết quả.

identifyTileRanges: dò biên, không binary search

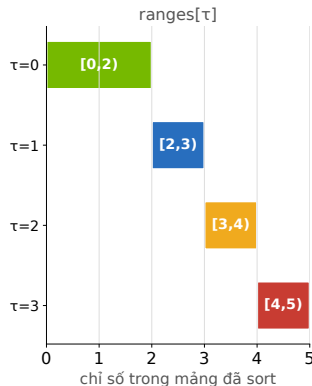


Không dùng binary search: mỗi thread so sánh cặp lân cận ($\text{idx} - 1, \text{idx}$), chi phí $O(1)$ mỗi thread.

- Mỗi thread so sánh $\tau(\text{idx})$ với $\tau(\text{idx} - 1)$; khi khác nhau thì ghi biên của hai tile.
- Biên đặc biệt: $\text{ranges}[\tau(0)].x = 0$ và $\text{ranges}[\tau(L - 1)].y = L$.
- Công việc $O(L)$, mỗi thread $O(1)$; tile không có Gaussian giữ $[0, 0)$ nhờ cudaMemset.

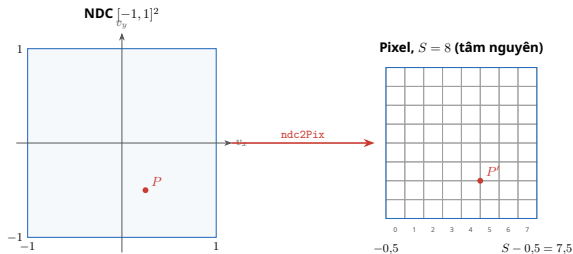
Ví dụ: A chạm 4 tile, B chạm 1 tile

vị trí	τ	d	bitcast(d)	key (hex 64 bit)	Gauss.
0	0	1.0	0x3F800000	0x000000003F800000	B
1	0	2.0	0x40000000	0x0000000040000000	A
2	1	2.0	0x40000000	0x0000000140000000	A
3	2	2.0	0x40000000	0x0000000240000000	A
4	3	2.0	0x40000000	0x0000000340000000	A



- Ảnh 32×32 , lưới 2×2 . A: $\mathbf{p} = (15, 15)$, $\Sigma = I$, $d = 2.0$, chạm $\tau = 0, 1, 2, 3$. B: $\mathbf{p} = (4, 4)$, $\Sigma = 0.25I$, $d = 1.0$, chạm $\tau = 0$.
- Sau sort, tile 0 có B trước A (gần trước xa), đúng thứ tự alpha-blending.
- $\text{ranges}[0] = [0, 2)$, $\text{ranges}[1] = [2, 3)$, $\text{ranges}[2] = [3, 4)$, $\text{ranges}[3] = [4, 5)$.

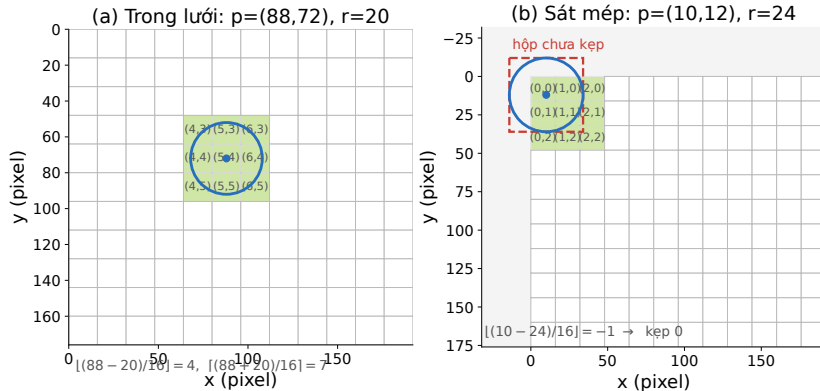
ndc2Pix: từ NDC sang tọa độ pixel



$$v = -1 \mapsto -0,5 \quad ; \quad v = 1 \mapsto S - 0,5 \quad ; \quad ndc2Pix(0,25) = \frac{1,25 \cdot 8 - 1}{2} = 4,5 \quad ; \quad P \mapsto P' = (4,5; 1,5)$$

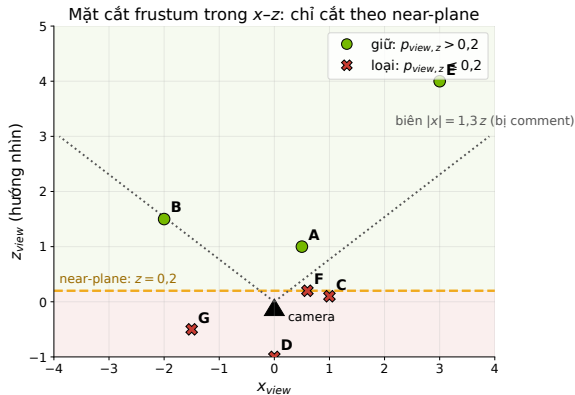
- $ndc2Pix(v, S) = \frac{(v + 1)S - 1}{2}$ với $v \in [-1, 1]$ (NDC), $S \in \{W, H\}$.
- Quy ước tâm pixel nguyên: $v = -1 \mapsto -0,5$ và $v = 1 \mapsto S - 0,5$.
- Trước đó: $(x', y', z', w') = M\bar{p}$ rồi $v = x'/w'$ (trong `in_frustum` dùng $w_{inv} = 1/(w' + 10^{-7})$).

getRect: hình chữ nhật tile bao quanh



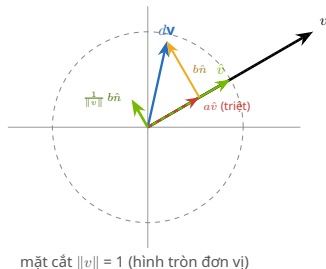
- $\text{rect}_{\min} = \lfloor (p - r) / B \rfloor$, $\text{rect}_{\max} = \lfloor (p + r + B - 1) / B \rfloor$ (làm tròn lên), $B = 16$.
- Kẹp về $[0, \text{grid}]$: Gaussian sát mép không sinh tile ngoài lưới (hình b).
- Ghi chú nguồn: getRect nằm trong đoạn duplicateWithKeys đã bị comment; đường chạy thật dùng processTiles.

in_frustum: cắt near-plane $z > 0,2$



- Điều kiện loại duy nhất đang chạy: $p_{view,z} \leq 0,2$ thì return false (auxiliary.h, dòng 167).
- Kiểm tra biên NDC $[-1,3; 1,3]^2$ bị *comment* trong code: hiện tại không loại theo x, y. Cone trong hình chỉ minh họa.
- p_{proj} được tính nhưng không dùng trong điều kiện cuối.

Chuẩn hoá vector và đạo hàm `dnormvdz`



Chuẩn hoá: $\hat{v} = \frac{v}{\|v\|}$, $s = \|v\|^2$

$$\frac{\partial \hat{v}_k}{\partial v_j} = \frac{s \delta_{kj} - v_k v_j}{s^{3/2}}$$

Dạng ma trận:

$$\frac{\partial \hat{v}}{\partial v} = \frac{1}{\|v\|} (I - \hat{v} \hat{v}^\top)$$

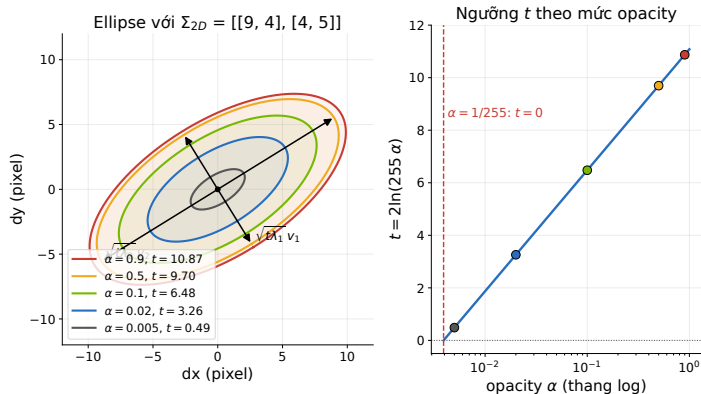
thành phần song song $\hat{v}$ bị triệt, vuông góc co theo $1/\|v\|$

`dnormvdz` (VJP, chỉ thành phần z):

$$\frac{-v_x v_z dv_x - v_y v_z dv_y + (s - v_z^2) dv_z}{s^{3/2}}$$

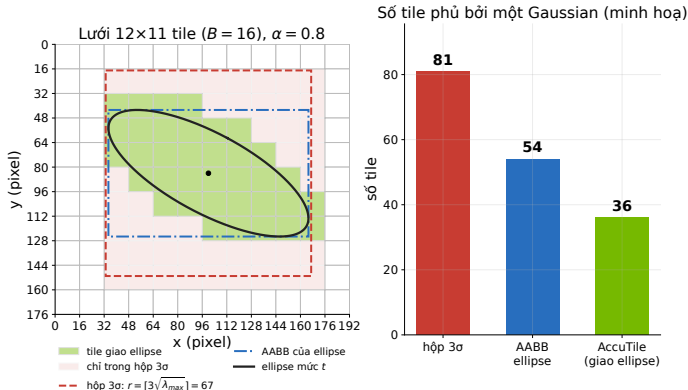
- $\hat{v} = v/\|v\|$, $s = \|v\|^2$. Đạo hàm: $\frac{\partial \hat{v}}{\partial v} = \frac{1}{\|v\|} (I - \hat{v} \hat{v}^\top)$.
- `dnormvdz` tính tích vector–Jacobian ngược, chỉ lấy thành phần z (khung đỏ).
- Thành phần song song với $\hat{v}$ bị triệt tiêu; `dnormvdx` là bản mở rộng cho quaternion 4 chiều.

Ellipse mức opacity: $t = 2 \ln(255 \alpha)$



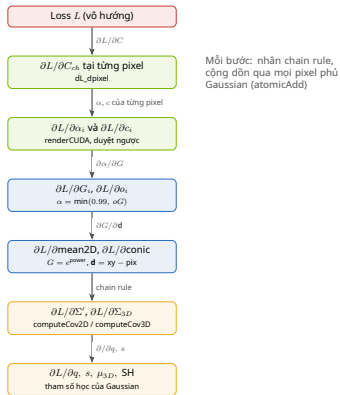
- Gaussian đóng góp $\geq 1/255$ khi $(x - p)^\top \Sigma^{-1}(x - p) \leq t$, với $t = 2 \ln(255 \alpha)$ (mult= 1).
- Trục ellipse: $\sqrt{t \lambda_i} v_i$, với (λ_i, v_i) là trị riêng và vector riêng của Σ_{2D} .
- Chỉ có ellipse khi $\alpha > 1/255$ (tức $t > 0$); code kiểm tra $a > 0, c > 0, b^2 - ac < 0$.

So sánh hộp 3σ , AABB và AccuTile



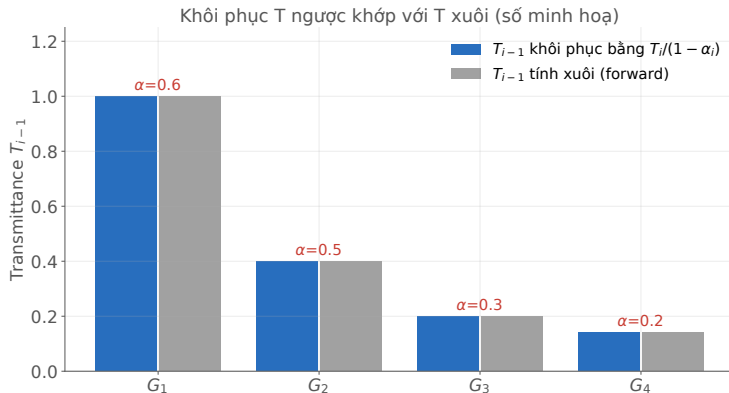
- Hộp 3σ (bản cũ): $r = \lceil 3\sqrt{\lambda_{\max}} \rceil = 67$, phủ **81** tile, nhiều tile ở góc không giao ellipse.
- AABB của ellipse: $\pm\sqrt{t\Sigma_{xx}}$, $\pm\sqrt{t\Sigma_{yy}}$, phủ **54** tile.
- `processTiles` (AccuTile) chỉ đếm tile thật sự giao ellipse: **36** tile.
- Số minh hoạ tự chọn: $\Sigma = \begin{pmatrix} 400 & 180 \\ 180 & 160 \end{pmatrix}$, $p = (100, 84)$, $\alpha = 0,8$ ($t \approx 10,6$).

Backward: đồ thị tính toán ngược



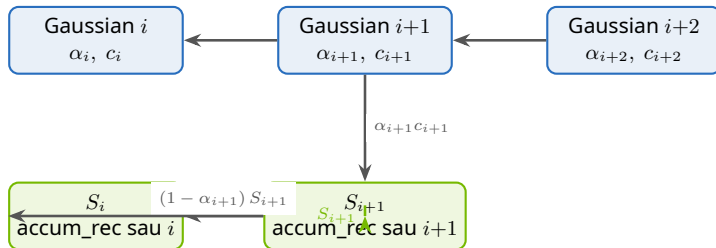
Đi từ loss L ngược về tham số: $\partial L / \partial C \rightarrow \partial L / \partial \alpha \rightarrow \partial L / \partial G \rightarrow \partial L / \partial \Sigma \rightarrow \partial L / \partial (q, s, \mu)$. Mỗi bước là chain rule trong backward.cu.

Khôi phục T bằng phép chia, không lưu lịch sử



Forward chỉ lưu T_{final} . Backward duyệt xa $\rightarrow$ gần, dùng $T_{i-1} = T_i/(1 - \alpha_i)$ (dòng $T = T / (1 - \alpha)$). Số trên hình là ví dụ minh hoạ.

Biến accum_rec: màu phía sau

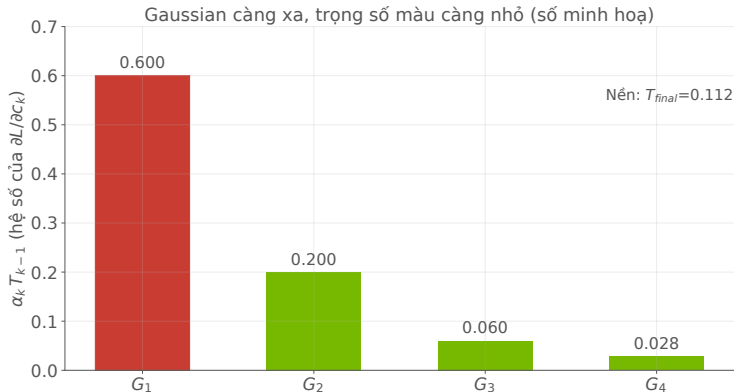


$$S_i = \alpha_{i+1} c_{i+1} + (1 - \alpha_{i+1}) S_{i+1}$$

$S_i = \alpha_{i+1} c_{i+1} + (1 - \alpha_{i+1}) S_{i+1}$ là dạng Horner của $C = \sum c_j \alpha_j T_{j-1}$. Gradient theo α_i :

$$\frac{\partial L}{\partial \alpha_i} = T_{i-1} \sum_{ch} (c_{i,ch} - S_{i,ch}) \frac{\partial L}{\partial C_{ch}}.$$

Gradient theo màu c_k



$\frac{\partial L}{\partial c_k} = \alpha_k T_{k-1} \frac{\partial L}{\partial C}$ (dchannel_dcolor = alpha * T, cộng bằng atomicAdd). Gaussian gần máy ảnh đóng góp nhiều nhất; nền T_{final} không có màu học được.

Bucket song song và ngưỡng bão hoà

renderCUDA (mục 1)

1 thread = 1 pixel

Duyệt Gaussian **ngược** (xa→gần)

Khôi phục $T_{i-1} = T_i / (1 - \alpha_i)$

Cộng $\partial L / \partial c_i$ bằng atomicAdd

PerGaussianRenderCUDA (mục 1.6)

1 warp (32 thread) = 1 Gaussian

Duyệt **xuôi** (gần→xa)

Checkpoint `sampled_ar`, `sampled_T` mỗi

32 Gaussian

Cùng công thức $\partial L / \partial \alpha$

Ngưỡng bão hoà (mục 1.7): nếu $o \cdot G > 0.99$ thì $\alpha = 0.99$ bị kẹp, bỏ qua gradient hình học
(continue sau khi đã cộng $\partial L / \partial c$ và $\partial L / \partial \alpha$)

Nguồn: backward.md, mục 1.6–1.7; so sánh chỉ về tổ chức song song, không đổi công thức

PerGaussianRenderCUDA đổi chiều song song (1 Gaussian = 1 warp) nhưng giữ công thức toán. Khi $o \cdot G > 0.99$, continue bỏ qua gradient hình học vì α bị kẹp.

Gaussian 2D: từ G đến vị trí pixel



$$\frac{\partial L}{\partial G} = o \frac{\partial L}{\partial \alpha} \quad \frac{\partial G}{\partial d_x} = -G(\text{conic}_x d_x + \text{conic}_{xy} d_y)$$

$$\frac{\partial G}{\partial d_y} = -G(\text{conic}_{xy} d_x + \text{conic}_y d_y)$$

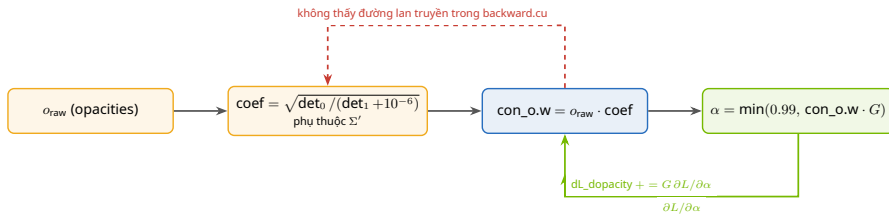
$$\frac{\partial L}{\partial \text{mean2D}_x} = \frac{\partial L}{\partial G} \frac{\partial G}{\partial d_x} \frac{W}{2}, \quad \frac{\partial L}{\partial \text{conic}_x} = -\frac{1}{2} d_x^2 G \frac{\partial L}{\partial G}$$

Đường đi ngược: $L \rightarrow \alpha \rightarrow G \rightarrow (\mathbf{d}, \text{conic})$; dL_dG và dG_ddelx trong renderCUDA

$\mathbf{d} = \text{xy}_{\text{screen}} - \text{pix}_f$ nên $\partial \mathbf{d} / \partial \text{xy} = I$. mean2D ở NDC nên có thêm hệ số $W/2$ (hoặc $H/2$). Gradient theo conic:

$$\partial L / \partial \text{conic}_x = -\frac{1}{2} d_x^2 G \partial L / \partial G.$$

Cần kiểm tra: opacity và hệ số bù khử alias

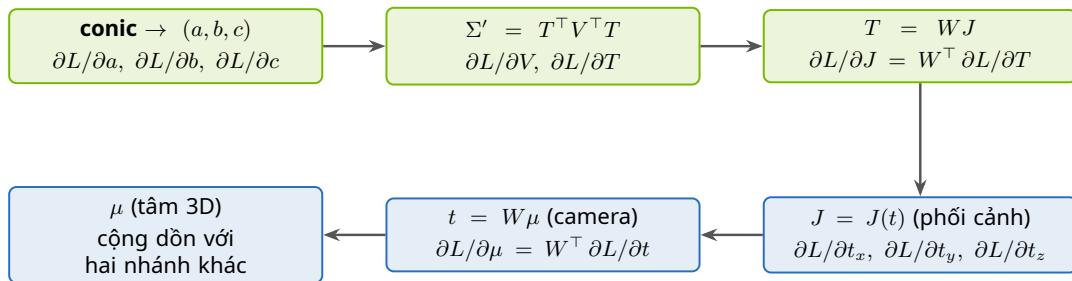


Cần kiểm tra, chưa khẳng định là lỗi:

- Trong backward.cu không có bước nhân coef khi cộng dL_{dopacity} .
- Không có gradient từ coef về Σ' , Σ_{3D} , scale, quaternion.
- Có thể là chủ đích (coi coef như stop-gradient, giống Mip-Splatting). Cần đối chiếu lớp Python gọi hàm này.

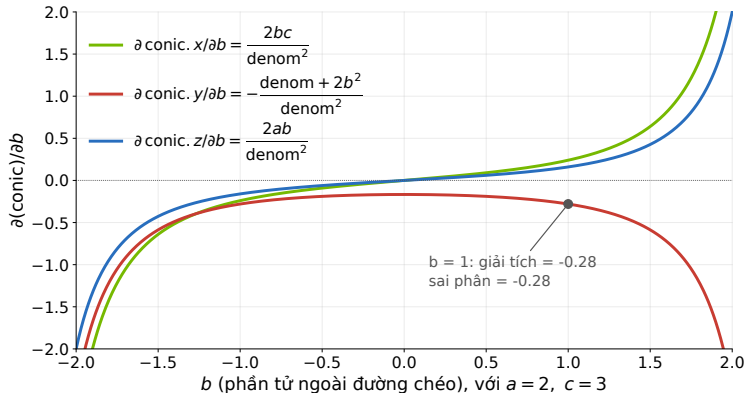
$\text{con_o.w} = \text{opacities} * \text{coef}$ với coef phụ thuộc Σ' . Trong backward.cu không thấy lan truyền qua coef.
Chưa khẳng định là lỗi; cần đối chiếu lớp Python và có thể là stop-gradient có chủ đích.

Backward qua Σ' , T , J và t về μ



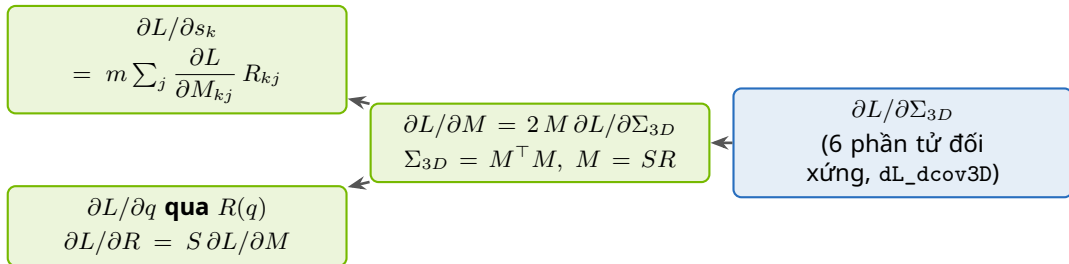
- $\Sigma' = T^\top V^\top T$ với $V = \Sigma_{3D}$; chỉ a, b, c được dùng nên gradient đi vào Σ' qua a, b, c .
- $T = WJ$ suy ra $\partial L / \partial J = W^\top \partial L / \partial T$; $J_{02} = -f_x t_x / t_z^2$.
- x_grad_mul, y_grad_mul bằng 0 khi $t_x / t_z, t_y / t_z$ bị kẹp; dL_dtz không có mask.
- Ghi = vào dL_dmeans tại `computeCov2DCUDA` (dòng 275); các nguồn khác cộng dồn +=.

Nghịch đảo 2×2 : từ conic đến a, b, c



- $\text{conic} = (c, -b, a) / \text{denom}$, $\text{denom} = ac - b^2$; $\partial L / \partial a = \frac{-c^2 g_x + 2bc g_y + (\text{denom} - ac) g_z}{\text{denom}^2}$, $g = \partial L / \partial \text{conic}$ (dòng 212).
- $\partial L / \partial b = \frac{2}{\text{denom}^2} (bc g_x - (\text{denom} + 2b^2) g_y + ab g_z)$ (dòng 214), đã khớp sai phân hữu hạn.
- **Lệch hằng số:** backward.cu (dòng 199, 201) dùng +0.3, forward.cu dùng 0.1.

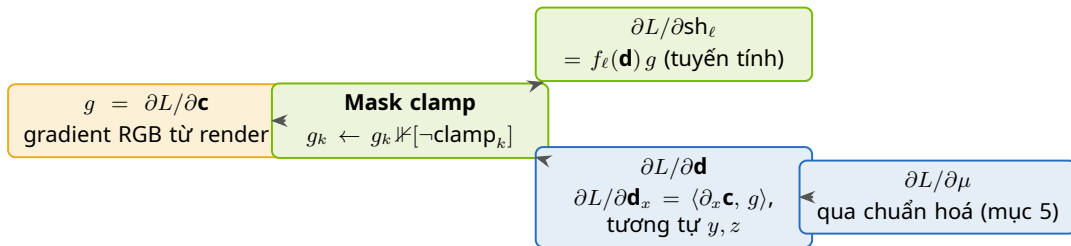
Backward computeCov3D: $\Sigma_{3D} \rightarrow M = SR \rightarrow s, q$



Không chuẩn hoá lại q : dòng `dnormvdv(rot, dL_dq)` bị comment (dòng 342), nên đạo hàm không đi qua $\|q\|$.

- Vì $\Sigma_{3D} = M^T M$ và $\partial L / \partial \Sigma_{3D}$ đối xứng: $\partial L / \partial M = 2 M \partial L / \partial \Sigma_{3D}$ (dòng 318).
- $M_{kj} = m s_k R_{kj}$: đúng là $\partial L / \partial s_k = m \sum_j \partial L / \partial M_{kj} R_{kj}$. Code (dòng 325–327) bỏ hệ số m , nên chỉ đúng khi $m = 1$.
- Không chuẩn hoá lại q : gradient qua $\|q\|$ bị bỏ, nên $\|q\| = 1$ phải đúng từ đầu vào.

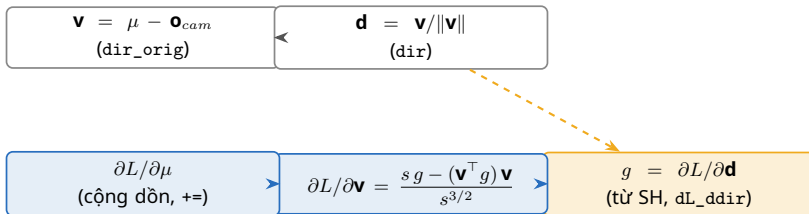
Backward SH: từ $\partial L / \partial \mathbf{c}$ đến sh_ℓ và hướng $\mathbf{d}$



- $\mathbf{c}(\mathbf{d}) = \sum_\ell f_\ell(\mathbf{d}) \text{sh}_\ell$ tuyến tính theo sh_ℓ , nên $\partial L / \partial \text{sh}_\ell = f_\ell(\mathbf{d}) g$ (dòng 49–58).
- Bậc 0: $f_{dc} = C_0 = \sqrt{1/4\pi}$; bậc 1: $f_0 = -C_1 y$, $f_1 = C_1 z$, $f_2 = -C_1 x$ với $C_1 = \sqrt{3/4\pi}$.
- Clamp: nếu kênh bị kẹp thì $g_k = 0$ (dòng 33–35); $\partial L / \partial \mathbf{d} = (\langle \partial_x \mathbf{c}, g \rangle, \dots)$ (dòng 132).

Chuẩn hoá hướng nhìn: dnormvdir và quy tắc thương

$$\begin{aligned} \text{Đạo hàm thương, } s &= \|\mathbf{v}\|^2: \\ \frac{\partial}{\partial v_j} \left(\frac{v_k}{\|\mathbf{v}\|} \right) &= \frac{\delta_{kj}}{\|\mathbf{v}\|} - \frac{v_k v_j}{\|\mathbf{v}\|^3} = \frac{s \delta_{kj} - v_k v_j}{s^{3/2}} \end{aligned}$$



- $\mathbf{v} = \mu - \mathbf{o}_{cam}$ (dir_orig, dòng 24) và $\mathbf{d} = \mathbf{v} / \|\mathbf{v}\|$ (dòng 25).
- Code (dòng 132): $\partial L / \partial v_x = ((s - v_x^2) dv_x - v_x v_y dv_y - v_x v_z dv_z) / s^{3/2}$, $s = \|\mathbf{v}\|^2$.
- Vì $\partial \mathbf{v} / \partial \mu = I$, gradient này được cộng trực tiếp vào $\partial L / \partial \mu$ (dòng 140).

Chain rule mean2D $\rightarrow \mu$ và cộng dồn ba nguồn

Nhánh Σ' (computeCov2DCUDA)

$$\partial L / \partial \mu|_{\Sigma} = W^{\top} \partial L / \partial t \quad (\text{ghi } =, \text{ dòng 275})$$

Nhánh mean2D (preprocessCUDA)

$$\partial L / \partial \mu_x = (P_{00} p_w - P_{30} p_{h,x} p_w^2) g_x + (P_{10} p_w - P_{30} p_{h,y} p_w^2) g_y \quad (+, \text{ dòng 391})$$

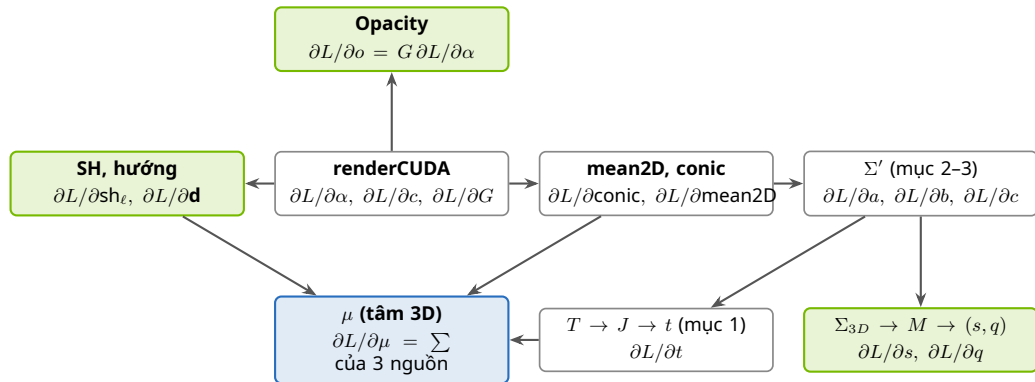
Nhánh SH (computeColorFromSH)

$$\partial L / \partial \mu = \text{dnormv}(\mu - \mathbf{o}_{cam}, \partial L / \partial \mathbf{d}) \quad (+, \text{ dòng 140})$$

$$\begin{aligned} \partial L / \partial \mu \text{ tổng} \\ = \sum \text{ của ba nhánh} \end{aligned}$$

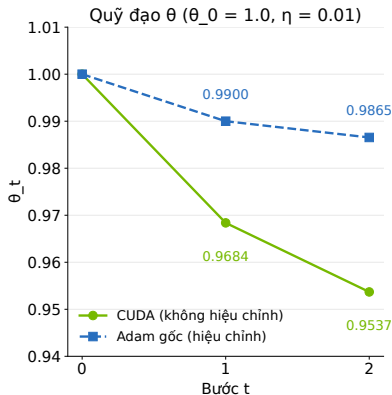
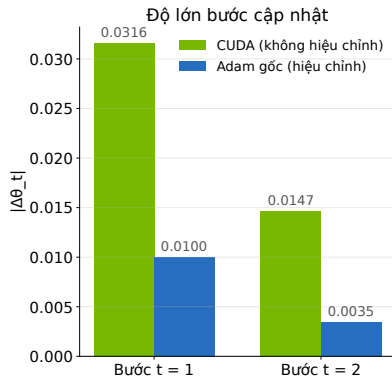
- mean2D = $(p_{h,x} p_w, p_{h,y} p_w)$, $p_h = P \tilde{\mu}$, $p_w = 1 / (p_{h,w} + \epsilon)$; đạo hàm chứa $-P_{30} p_{h,x} p_w^2$ từ phép chia phối cảnh (dòng 383–387).
- μ ảnh hưởng L qua ba đường: Σ' (qua t), vị trí chiếu mean2D, và hướng nhìn của SH.
- Các nguồn cộng dồn theo nguyên lý chain rule đa biến: dL_dmeans nhận += từ mean2D (dòng 391) và SH (dòng 140).

Tổng hợp một vòng backward: từ loss đến μ , s , q , SH, opacity



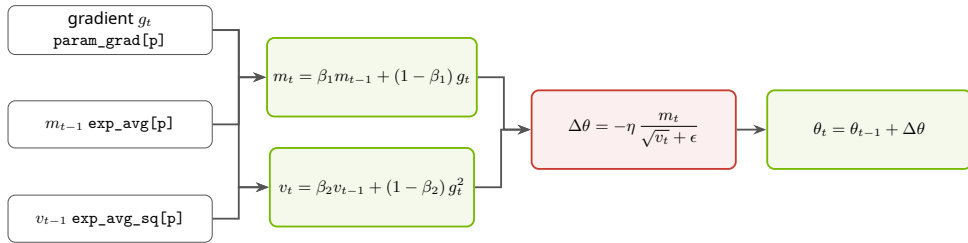
- Nguồn: backward.cu có trong repo tại `submodules/diff-gaussian-rasterization_structgs/cuda_rasterizer/`; mọi công thức lấy từ `backward.md`.
- $\partial L / \partial o = G \partial L / \partial \alpha$ (dòng 781), với o là `con_o.w` = $o_{raw} \cdot coef$; gradient **không** lan truyền qua hệ số bù coef.

Adam gốc và Adam trong CUDA: khác nhau ở bias correction



- Adam gốc (Kingma & Ba) dùng $\hat{m}_t = m_t / (1 - \beta_1^t)$ và $\hat{v}_t = v_t / (1 - \beta_2^t)$.
- Kernel `adamUpdateCUDA` không có hệ số hiệu chỉnh này: dùng trực tiếp m_t , v_t .
- Ví dụ trong tài liệu ($g_1 = 0.5$, $g_2 = -0.2$): $\theta_2 = 0.95369$ theo CUDA, $\theta_2 = 0.98654$ theo Adam gốc.

Công thức cập nhật trong adamUpdateCUDA



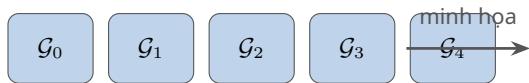
Không có $\hat{m}_t = m_t / (1 - \beta_1^t)$ và $\hat{v}_t = v_t / (1 - \beta_2^t)$

$$\beta_1 = 0.9, \quad \beta_2 = 0.999, \quad \epsilon = 10^{-8}$$

- Kernel đọc g_t từ param_grad, cập nhật m và v rồi ghi θ tại chỗ.
- SparseGaussianAdam.step gọi $\beta_1 = 0.9$, $\beta_2 = 0.999$ trực tiếp, không đọc group["betas"].
- $\epsilon = 10^{-8}$ mặc định, lấy từ adam_eps_order = 8; η lấy từ group["lr"].

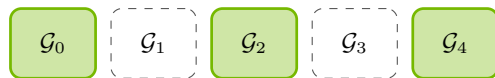
Cập nhật thưa theo visibility: `tiles_touched[g]`

Adam dày đặc
(`torch.optim.Adam`)



Mọi N Gaussian đều đi qua bước cập nhật. Gradient bằng 0 vẫn làm m, v suy giảm.

Sparse (adamUpdateCUDA)
kiểm tra `tiles_touched[g]`



Chỉ Gaussian visible được cập nhật (θ, m, v). Gaussian ẩn giữ nguyên hoàn toàn.

- Python truyền mặt nạ `visibility`; kernel nhận đối số `tiles_touched`.
- Mặt nạ tra theo Gaussian $g = \lfloor p/M \rfloor$, nên cả M thành phần của một Gaussian được cập nhật hoặc bỏ qua cùng lúc.
- Gaussian không visible giữ nguyên θ, m, v ; không bị suy giảm như khi coi $g = 0$.

Cầu nối PyTorch $\leftrightarrow$ CUDA: `ext.cpp` và `module _C`



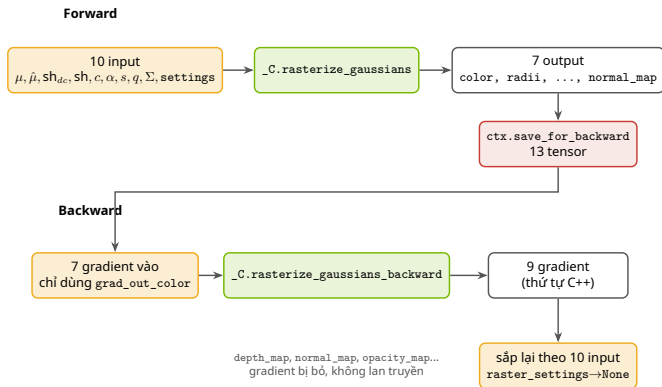
Bốn hàm được export từ `ext.cpp`:

```
rasterize_gaussians rasterize_gaussians_backward  
mark_visible adamUpdate
```

Lớp glue: kiểm tra shape, cấp phát buffer, không chứa công thức toán mới

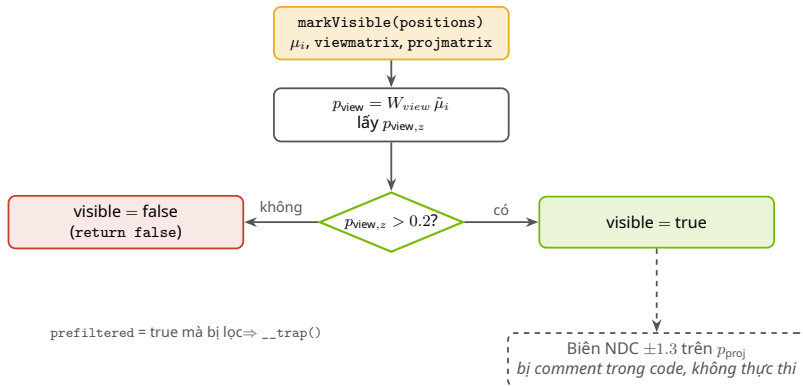
- `ext.cpp` dùng `PYBIND11_MODULE(TORCH_EXTENSION_NAME, m)` để export 4 hàm vào `module _C`.
- `rasterize_points.cu` chỉ kiểm tra shape, cấp phát buffer và chuyển con trỏ; không có công thức toán mới.
- `rasterize_points.h` khai báo `conv2DForward` nhưng `ext.cpp` không đăng ký hàm này.

Autograd: `_RasterizeGaussians` forward và backward



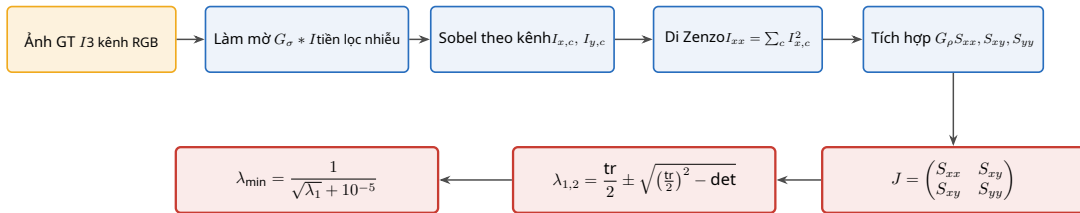
- Forward nhận 10 input, gọi `_C.rasterize_gaussians`, trả 7 output và lưu 13 tensor cho backward.
- Backward chỉ dùng `grad_out_color`; gradient của `depth_map`, `normal_map` không lan truyền.
- `raster_settings` không phải tensor nên gradient trả về là `None`; `means2D` là placeholder để nhận gradient.

markVisible và frustum: chỉ lọc near-plane



- Điều kiện thực thi trong `auxiliary.h` (dòng 167): $p_{view.z} \leq 0.2f$ thì loại Gaussian.
- Điều kiện biên NDC ± 1.3 trên p_{proj} đã bị comment; Gaussian ngoài trường nhìn vẫn được giữ.
- `markVisible` bọc trong `torch.no_grad()` và gọi `_C.mark_visible`.

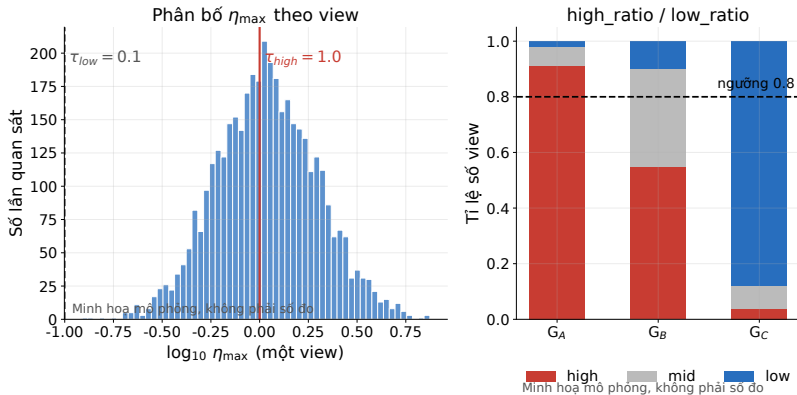
SADGS (1/7): structure tensor đa tỉ lệ từ ảnh GT



$\lambda_{\min}$: bước sóng nhỏ nhất (px) Gaussian còn đọc được; λ_1 : năng lượng gradient bình phương.

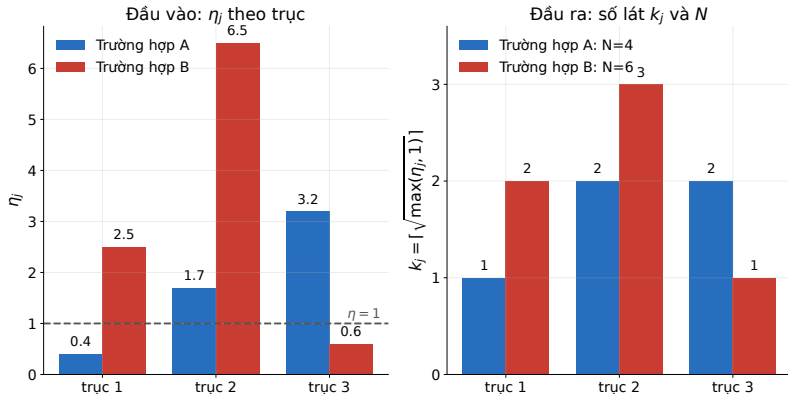
- Tensor cấu trúc J chỉ phụ thuộc ảnh GT, tính một lần trước huấn luyện (`train.py`), trong `torch.no_grad()`.
- Đa tỉ lệ: $\sigma_i = 1.5^i$, $\rho_i = 3\sigma_i$, gộp theo trọng b_i^3 (`power_factor=3`).
- $\lambda_{\min} = 1/(\sqrt{\lambda_1} + 10^{-5})$ là bước sóng nhỏ nhất còn nhìn thấy, đơn vị pixel.

SADGS (2/7): η theo từng trục và phiếu bầu qua view



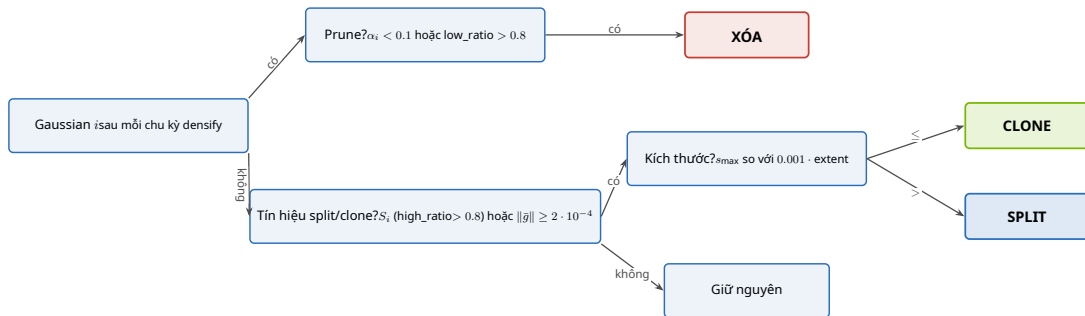
- $\eta_j = \|a_j\| / \lambda_{\min}$, với $\|a_j\|$ là độ dài trục j của splat sau khi chiếu (pixel). Tỉ số pixel/pixel nên η không thứ nguyên.
- Mỗi view phân loại theo $\eta_{\max} = \max_j \eta_j$: high nếu $> \tau_{\text{high}} = 1.0$, low nếu $\leq \tau_{\text{low}} = 0.1$.
- $\text{high_ratio} = \frac{\text{eta_high_count}}{\text{accum_view_count}}$, tương tự cho low_ratio.

SADGS (3/7): split dị hướng, $k_j = \lceil \sqrt{\max(\eta_j, 1)} \rceil$



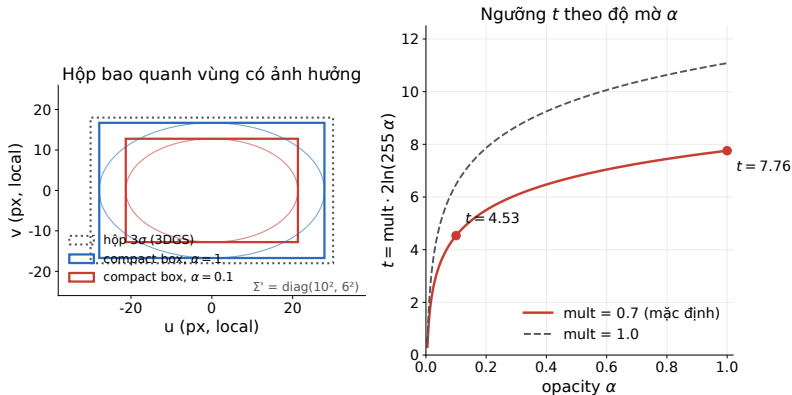
- Điều kiện lấy mẫu: $\sigma'_a \omega_{\max} \leq 1$ với $\sigma'_a = \sigma_a / k_a$, suy ra $k_a \geq \sqrt{\eta_a}$.
- Số con $N = k_x k_y k_z$ khác nhau theo từng Gaussian. 3DGS gốc luôn chia cố định 2 con đẳng hướng.
- Ví dụ: trường hợp A cho $N = 4$, trường hợp B cho $N = 6$ (số minh họa).

SADGS (4/7): clone và prune theo cây quyết định



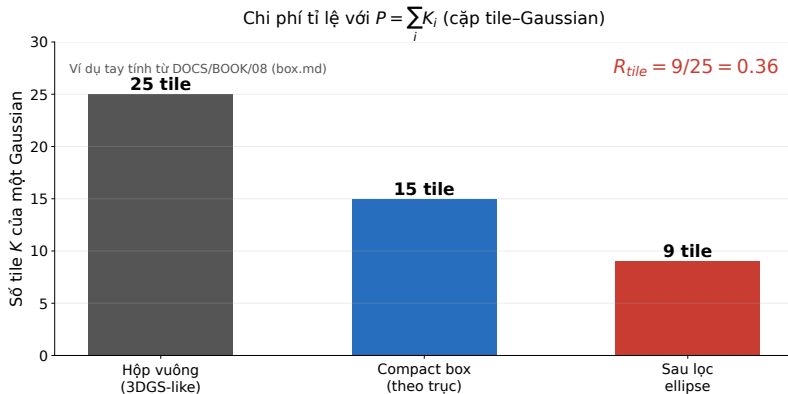
- Prune: $\alpha_i < 0.1$ (min_opacity, gấp 20 lần mặc định 3DGS 0.005), hoặc low_ratio > 0.8 khi đã có view.
- Clone khi $s_{\max} \leq 0.001 \cdot \text{extent}$; split khi lớn hơn. dense = 0.001.

SADGS (5/7): compact box, $t = \text{mult} \cdot 2 \ln(255 \alpha)$



- $\text{half}_x = \sqrt{t \Sigma'_{11}}$, $\text{half}_y = \sqrt{t \Sigma'_{22}}$, với $\text{mult} = 0.7$ mặc định.
- Mặc định: $\alpha = 1 \Rightarrow t \approx 7.76$; $\alpha = 0.1 \Rightarrow t \approx 4.53$. Với $\text{mult} = 1$: 11.08 và 6.48.
- Hình minh họa với $\Sigma' = \text{diag}(10^2, 6^2)$ px, không phải số đo từ cảnh.

SADGS (6/7): chi phí tỉ lệ với $P = \sum_i K_i$



- Kernel trả chi phí theo cặp (tile, Gaussian): $P = \sum_i K_i$, K_i là số tile Gaussian i chạm tới.
- Ví dụ tay tính: $25 \rightarrow 15 \rightarrow 9$ tile, nên $R_{tile} = 9/25 = 0.36$.
- Diện tích tiệm cận 0.121 nhưng đo được 0.36, vì lưới tile làm tròn lên (số hạng +1).

Digital twin trạm BTS: ứng dụng, hạn chế, chi phí

Ứng dụng digital twin

- Kiểm tra anten và thiết bị trên cột
- Xem từ góc nhìn mới, không cần leo cột
- So sánh trước và sau bảo trì trên cùng mô hình

Hạn chế

- Ánh sáng thay đổi giữa các lần chụp
- Kim loại phản chiếu gây sai lệch màu
- Thiếu góc chụp: vùng khuất không được tái tạo

2 968 520
Gaussian cuối

7.21 GB
VRAM đỉnh

4234.8 s
thời gian train

Số liệu chạy thật: scene HCM0539, 30 000 iter (DOCS/README.md).

- Chi phí GPU: 2 968 520 Gaussian, VRAM đỉnh 7.21 GB, 4234.8 s (khoảng 1h10p) cho HCM0539 ở 30 000 iter.
- Hạn chế về ánh sáng, phản chiếu kim loại và góc chụp là mức khái niệm, không có số đo riêng.

Cảm ơn — Hỏi đáp

Nguồn: `MATH/submodules/diff-gaussian-rasterization_structgs/`, `DOCS/BOOK/`, `bts_digital_twin.ipynb`